1. Merge Sort

Divide → Conquer → Combine

- Divide: Split the list into smaller halves.
- Base Case: Stop when the list has **0 or 1 item**.
- Conquer: Sort the smaller sublists.
- Combine: Merge the sorted sublists back together.

Key Things to Remember

- Uses **Divide-and-Conquer**.
- Merge step combines **two sorted lists** into one sorted list.
- Time Complexity: **O(n log n)**.
- Considered **stable** (equal values keep their relative order).

---

2. Quick Sort

PIVOT

Key Things to Remember

- Pivot divides values into:
    - Less than pivot
    - Greater than pivot
- Partitioning is based on comparing values against the pivot.
- After partitioning:
    - The pivot is already in its **final sorted position**.
- Recursive calls continue on the left and right partitions.
- Average Case: **O(n log n)**
- Worst Case: **O(n^2)**

Quick Sort is often described as: **In-place sorting**

Meaning: Rearranges items inside the original array with little extra memory.

---

3. Merge Sort vs Quick Sort

Merge Sort	Quick Sort
Split into halves	Partition around a pivot
Merge sorted lists	Partition left/right of pivot
Stable	Not inherently stable
O(n log n) consistently	Average O(n log n), Worst $O(n^2)$

---

4. Array-Based Sequences

Key Ideas

Python lists are **dynamic arrays**

Meaning:

- Can grow
- Can shrink

Python uses **zero-based indexing**

Item	Index
First Item	0
Second Item	1
Last Item	-1

Why is indexing fast?

Because items are stored in **contiguous memory**.

Direct access → O(1)

Operations to Remember

Operation	Efficiency
Access A[i]	Fast
Append to end	Usually fast

Insert at front	Slow
Remove first item	Slow

Why slow? Because many items may need to **shift**.

---

5. Dynamic Arrays

Key Concept

A dynamic array has:

- Size (number of items)
- Capacity (available storage)

When full:

1. Create a larger array
2. Copy existing items
3. Continue inserting

"Why allocate more capacity than needed?"

Answer:

To support future appends efficiently without resizing every time.

---

6. Stacks

Principle

LIFO

Last-In, First-Out

Operations

Operation	**Meaning**
push	Add to top
pop	Remove from top
peek	View top without removing
isEmpty	Check whether stack has items

Real-World Example

Undo operation in software.

---

7. Queues

Principle

FIFO

First-In, First-Out

Operations

Operation	Meaning
enqueue	Add at back
dequeue	Remove from front

Real-World Example

Printer queue

---

8. Circular Queue

Key Idea

When the rear reaches the last index:

It wraps around to index 0.

Benefit:

- Reuses empty space.
- Avoids shifting items.

---

9. ADT (Abstract Data Type)

An ADT describes:

What operations are available NOT How the structure is implemented

Example for a Queue ADT, the following describe behaviour:

- enqueue(item)
- dequeue()